AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

# DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

## Algorytm listy dwukierunkowej

Autor:
Jakub Pyrdoł

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

# Spis treści

**1. Ogólne określenie wymagań** **3**

1.1. Przykład . . . . . 3

1.2. Instalacja . . . . . 3

**2. Analiza problemu** **4**

2.1. Reprezentacja i sposób przechodzenia po liście . . . . . 4

2.2. Tworzenie węzłów: Factory . . . . . 4

2.3. Przechodzenie po elementach: Iterator . . . . . 4

**3. Projektowanie** **5**

3.1. Język i organizacja kodu . . . . . 5

3.2. Podział odpowiedzialności . . . . . 5

3.3. Diagram klas . . . . . 6

**4. Implementacja** **7**

4.1. Klasa Node . . . . . 7

4.2. Klasa Factory . . . . . 7

4.3. Klasa Iterator . . . . . 8

4.4. Interfejs klasy DoublyLinkedList . . . . . 9

4.5. Wyniki działania . . . . . 10

**5. Wnioski** **11**

**Literatura** **12**

**Spis rysunków** **13**

**Spis tabel** **14**

**Spis listingów** **15**

# 1. Ogólne określenie wymagań

Celem projektu jest opracowanie listy dwukierunkowej w języku C++. Przechowuje ona liczby całkowite. Każdy węzeł zawiera wartość i wskaźniki na obu sąsiadów. Reprezentację danych, tworzenie węzłów i przechodzenie po kolekcji rozdzielono między klasy `Node`, `Factory` i `Iterator`. Interfejs zarządzania listą udostępnia klasa `DoublyLinkedList`.

Opis oparto na dostarczonej dokumentacji Doxygen [1]. Zawiera ona deklaracje czterech klas oraz opis działania konstruktora węzła, fabryki i iteratora. Dla klasy `DoublyLinkedList` pokazano nagłówek, bez definicji metod. Operacje tej klasy opisano więc jako zadeklarowany interfejs, bez przypisywania im wyników działania.

Udokumentowany interfejs obejmuje:

- dodawanie elementu na początku, na końcu i pod wskazanym indeksem;
- usuwanie elementu z początku, z końca i z wybranej pozycji;
- wyświetlanie listy w obu kierunkach oraz udostępnienie metod wyświetlających następny i poprzedni element;
- czyszczenie listy;
- uzyskiwanie iteratorów przez metody `begin()` i `end()`.

Parametr `value` ma typ `int`. Parametr `index` oznacza pozycję liczoną od zera.

## 1.1. Przykład

Dla listy zawierającej kolejno wartości 10, 20 i 30 model połączeń można zapisać następująco:

$$\underbrace{[10]}_{\texttt{head}} \rightleftarrows [20] \rightleftarrows \underbrace{[30]}_{\texttt{tail}}.$$

Pole `next` węzła o wartości 20 prowadzi do węzła o wartości 30, a pole `prev` do węzła o wartości 10. Przejście po `next` daje kolejność 10, 20, 30, a przejście od końca po `prev` kolejność 30, 20, 10. Jest to przykład objaśniający strukturę danych.

## 1.2. Instalacja

# 2. Analiza problemu

Lista dwukierunkowa reprezentuje uporządkowany ciąg elementów z połączeniami w obu kierunkach. Jest przydatna w kolekcjach wymagających przechodzenia między sąsiadami i modyfikowania ich połączeń. W projekcie każdy węzeł przechowuje liczbę całkowitą.

## 2.1. Reprezentacja i sposób przechodzenia po liście

Klasa `Node` zawiera pola `value`, `next` i `prev`. Klasa `DoublyLinkedList` przechowuje wskaźniki `head` i `tail`, obiekt `factory` oraz pole `size` typu `int`. Wskaźniki na początek i koniec określają punkty rozpoczęcia przejścia.

Pole `next` prowadzi do następnego węzła, a `prev` do poprzedniego. Wstawienie węzła między dwa elementy wymaga połączenia go z oboma sąsiadami, a usunięcie węzła ponownego połączenia jego sąsiadów. Jest to zasada działania struktury; definicji tych operacji nie pokazano w dostarczonym PDF-ie.

## 2.2. Tworzenie węzłów: Factory

Metoda `Factory::createNode(int value)` zwraca wskaźnik `Node*`. Zgodnie z dokumentacją korzysta z operatora `new`, tworzy węzeł na stercie i przekazuje mu otrzymaną wartość. Oddziela to tworzenie elementów od zarządzania połączeniami w klasie listy.

Konstruktor `Node(int value)` według opisu przypisuje liczbę do pola `value`, a pola `next` i `prev` ustawia na `nullptr`. Nowy węzeł nie jest jeszcze połączony z pozostałymi elementami.

## 2.3. Przechodzenie po elementach: Iterator

W klasie `Iterator` prywatne pole `current` wskazuje bieżący węzeł. Konstruktor przyjmuje wskaźnik początkowy. Operator `++` przesuwa iterator przez `next`, a operator `--` przez `prev`. Operator `*` zwraca wartość `int` z bieżącego węzła, a `!=` porównuje pozycje dwóch iteratorów.

Metoda `begin()` ma zwracać iterator na początku, a `end()` pozycję za ostatnim elementem. Dokumentacja wskazuje dla tej drugiej pozycji typowe użycie `nullptr`, bez pokazania definicji metody. Nie określa zachowania iteratora poza granicami listy.

# 3. Projektowanie

## 3.1. Język i organizacja kodu

Projekt wykorzystuje język C++. Opis kodu wygenerowano przy użyciu programu Doxygen 1.18.0. Zestawienie plików widocznych w tym opisie przedstawia tabela 3.1.

**Tab. 3.1.** Pliki wymienione w dokumentacji Doxygen

| **Klasa** | **Pliki** |
|---|---|
| `Node` | `Node.h`, `Node.cpp` |
| `Factory` | `Factory.h`, `Factory.cpp` |
| `Iterator` | `Iterator.h`, `Iterator.cpp` |
| `DoublyLinkedList` | `DoublyLinkedList.h` |

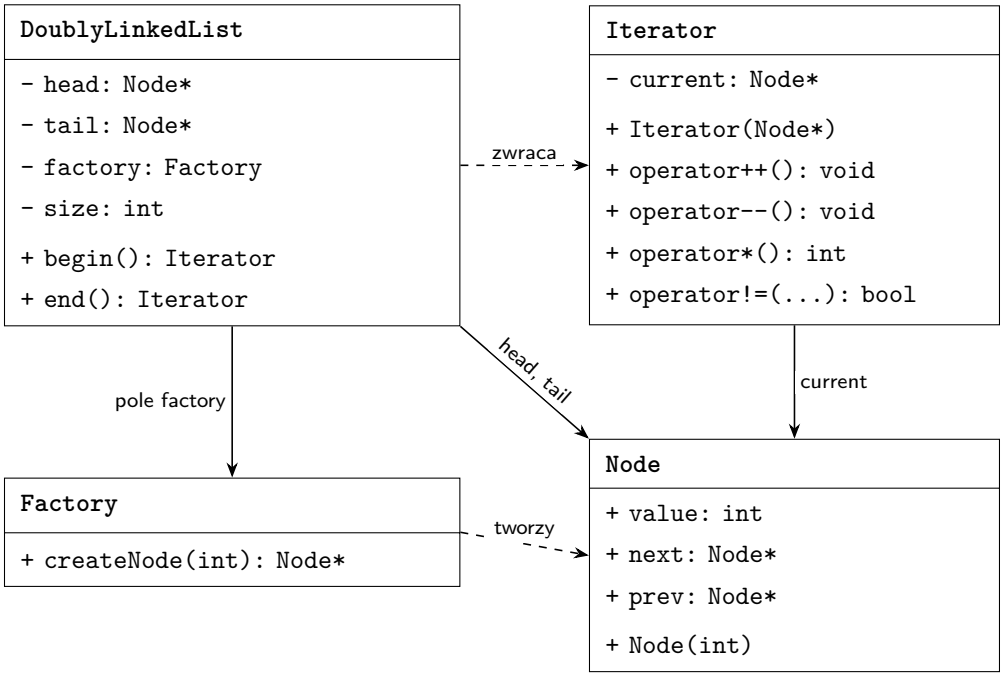
## 3.2. Podział odpowiedzialności

Klasa `Node` reprezentuje pojedynczy element i jego połączenia. Klasa `Factory` tworzy węzły. Klasa `Iterator` przechowuje aktualną pozycję i udostępnia operatory przejścia oraz odczytu. Klasa `DoublyLinkedList` udostępnia interfejs zarządzania całą strukturą.

W klasie listy pola `head`, `tail`, `factory` i `size` są prywatne. W iteratorze prywatny jest wskaźnik `current`. Pola węzła są publiczne. Jest to podział widoczny bezpośrednio w deklaracjach klas.

## 3.3. Diagram klas

Rysunek 3.1 przedstawia zależności wynikające z pól klas, dołączanych nagłówków i typów zwracanych przez metody. Na diagramie umieszczono pola oraz wybrane metody; pełny interfejs listy znajduje się w rozdziale 4. Strzałki opisują użycie obiektów i wskaźników, bez zakładania dziedziczenia.




**Rys. 3.1.** Klasy i zależności widoczne w udokumentowanym interfejsie

# 4. Implementacja

Poniższe listingi odtwarzają deklaracje klas widoczne w dokumentacji Doxygen. Pominięto dyrektywy ochronne nagłówków i komentarze, zachowując pola, typy, widoczność oraz sygnatury metod.

## 4.1. Klasa Node

Węzeł przechowuje liczbę całkowitą i dwa wskaźniki do obiektów tego samego typu. Publiczny konstruktor przyjmuje wartość początkową. Według jego opisu pola połączeń są inicjalizowane wartością `nullptr`.

```
class Node
{
public:
    int value;
    Node* next;
    Node* prev;

    Node(int value);
};
```

**Listing 1.** Deklaracja klasy Node z dokumentacji kodu

## 4.2. Klasa Factory

Metoda `createNode` przyjmuje wartość `int` i zwraca wskaźnik do nowo utworzonego węzła. Dokumentacja opisuje dynamiczną alokację za pomocą `new`. Nie przedstawia dodatkowych mechanizmów obsługi błędu alokacji.

```
#include "Node.h"

class Factory
{
public:
    Node* createNode(int value);
};
```

**Listing 2.** Deklaracja klasy Factory z dokumentacji kodu

## 4.3. Klasa Iterator

Iterator jest inicjalizowany wskaźnikiem do węzła. Operatory `++` i `--` w widocznym interfejsie nie przyjmują argumentów i zwracają `void`. Operator wyłuskania zwraca liczbę `int`, a porównanie `!=` wartość `bool`.

```
#include "Node.h"

class Iterator
{
private:
    Node* current;

public:
    Iterator(Node* current);
    void operator++();
    void operator--();
    int operator*();
    bool operator!=(const Iterator& other);
};
```

**Listing 3.** Deklaracja klasy Iterator z dokumentacji kodu

Operator `!=` porównuje wskazywane węzły, a nie przechowywane w nich liczby. Dwa różne węzły mogą zatem przechowywać tę samą wartość, pozostając różnymi pozycjami w strukturze.

## 4.4. Interfejs klasy DoublyLinkedList

Klasa listy łączy wskaźniki początku i końca z obiektem fabryki oraz polem `size`. Listing 4 przedstawia pełny zestaw deklaracji widoczny w udostępnionym nagłówku.

```
#include "Factory.h"
#include "Iterator.h"
#include "Node.h"

class DoublyLinkedList
{
private:
    Node* head;
    Node* tail;
    Factory factory;
    int size;

public:
    void push_front(int value);
    void push_back(int value);
    void push_wherever(int index, int value);

    void pop_front();
    void pop_back();
    void pop_wherever(int index);

    void display_list();
    void display_reversed_list();
    void display_next_element();
    void display_prev_element();
    void clear_list();

    Iterator begin();
    Iterator end();
};
```

**Listing 4.** Deklaracja klasy DoublyLinkedList z dokumentacji kodu

Znaczenie poszczególnych operacji podane w dokumentacji Doxygen zebrano w tabeli 4.1. Jest to opis zadeklarowanego interfejsu, a nie wynik testów programu.

**Tab. 4.1.** Operacje zadeklarowane w klasie DoublyLinkedList

| **Metoda** | **Znaczenie według dokumentacji** |
|---|---|
| `push_front(int value)` | Dodanie węzła na początku listy. |
| `push_back(int value)` | Dodanie węzła na końcu listy. |
| `push_wherever(int index, int value)` | Wstawienie wartości pod wskazanym indeksem, liczonym od 0. |
| `pop_front()` | Usunięcie pierwszego węzła i zwolnienie jego pamięci. |
| `pop_back()` | Usunięcie ostatniego węzła i zwolnienie jego pamięci. |
| `pop_wherever(int index)` | Usunięcie elementu pod indeksem liczonym od 0. |
| `display_list()` | Wyświetlenie listy od początku do końca. |
| `display_reversed_list()` | Wyświetlenie listy od końca do początku. |
| `display_next_element()` | Wyświetlenie następnego elementu w sekwencji. |
| `display_prev_element()` | Wyświetlenie poprzedniego elementu w sekwencji. |
| `clear_list()` | Usunięcie wszystkich węzłów i zwolnienie ich pamięci. |
| `begin()` | Zwrócenie iteratora wskazującego początek listy. |
| `end()` | Zwrócenie iteratora reprezentującego pozycję za ostatnim elementem. |

## 4.5. Wyniki działania

# 5. Wnioski

# Literatura

[1] ListaDwukierunkowa. dokumentacja kodu. Dokumentacja wygenerowana przez Doxygen 1.18.0. Materiał dostarczony przez autora projektu w pliku doc(1).pdf; opis klas na stronach 5–12, deklaracje nagłówków na stronach 13–15.

# Spis rysunków

3.1. Klasy i zależności widoczne w udokumentowanym interfejsie . . . . . 6

# Spis tabel

3.1. Pliki wymienione w dokumentacji Doxygen . . . . . 5

4.1. Operacje zadeklarowane w klasie DoublyLinkedList . . . . . 10

# Spis listingów

1. Deklaracja klasy Node z dokumentacji kodu . . . . . 7
2. Deklaracja klasy Factory z dokumentacji kodu . . . . . 7
3. Deklaracja klasy Iterator z dokumentacji kodu . . . . . 8
4. Deklaracja klasy DoublyLinkedList z dokumentacji kodu . . . . . 9